S02 - Ashokavana: The Veiled Sanctuary

Flag: `PINAKA{setu_bandhan_veiled_sanctuary_fused_recon_and_rations}` Validator: `https://pinaka-s02-cyber-xo-x.vercel.app/sanctuary/verify`

Overview

Two independent puzzles that have to agree. A damaged aerial survey narrows the candidate gardens down to a family of look-alikes. A provisions ledger plus a road graph with one broken edge gives a distance, and only one garden from that family is at that distance. The answer is Garden 2, Shimshapa Bower.

The survey

`recon.survey`: magic `PNKVEIL1`, version 1, 6 images, 7 stones. Both the header and calibration-table checksums are valid, but the 64-byte calibration table itself is all zeros. So the detector matrix has to be recovered from the metric anchor image. It comes out as exact fractions:

```
D = [[ 6/5,  1/10],
     [-1/20, 11/10]]
```

Some frames are torn, so I dropped them before fitting. I applied the inverse detector matrix to the good observations and checked every stone against each candidate template. Gardens `[1, 2, 4]` match. They share one canonical template, whose hash is `feec6a12bc90a538ec4ca3a72b74f62ae0bc4579c1d3a41cdc7706aba994cec4`.

The ledger

The published road lengths add up to 75, but the royal total is 79, so the sentinel edge `(2, 3)` must be 4. With that fixed, Dijkstra from depot 0 puts garden 2's road node at distance 14. The rations give the same number on their own: `(50 - 8) / 3 = 14` leagues. Among gardens 1, 2 and 4, only garden 2 is at 14.

Submitted: garden 2, repaired edge `[2,3,4]`, ledger distance 14, garden distance 14.

Result

The validator answered `sanctuary-confirmed`:

```
PINAKA{setu_bandhan_veiled_sanctuary_fused_recon_and_rations}
```

```
/home/darknotch/CTF/Pinaka/S02_player
Permissions Size User      Date Modified Name
.rw-rw-r--  3.0k darknotch 30 Oct 07:05  {} city_model.json
.rw-rw-r--  1.5k darknotch 30 Oct 07:05  📄 recon.survey
.rw-rw-r--   726 darknotch 30 Oct 07:05  {} supply_records.json

…/Pinaka/S02_player ❯
```

```
Survey container: 1520 bytes; magic/header/table CRCs verified; detector table is zeroed.
Recovered detector: {"d00": [6, 5], "d01": [1, 10], "d10": [-1, 20], "d11": [11, 10]}
Authentic family: [1, 2, 4] template digest: feec6a12bc90a538ec4ca3a72b74f62ae0bc4579c1d3a41cdc7706aba994cec4
Repaired edge: [2, 3, 4] ledger distance: 14 candidate: 2 distance: 14

…/Pinaka/S02_player ❯
```


```
    ]
  },
  "template_digest": "feec6a12bc90a538ec4ca3a72b74f62ae0bc4579c1d3a41cdc7706aba994cec4",
  "authentic_family": [
    1,
    2,
    4
  ],
  "repaired_edge": [
    2,
    3,
    4
  ],
  "ledger_distance": 14,
  "garden_distance": 14
}

…/Pinaka/S02_player ❯
```





Solver

`solve.py` checks the container (magic, counts, both CRCs, the zeroed calibration block), recomputes the family and the template digest from `city_model.json` using the recovered matrix, fixes the road graph and writes `submission.json`. `--submit` sends it.

```
#!/usr/bin/env python3
"""Rebuild the S02 proof payload from the supplied survey and city/ledger files."""
from __future__ import annotations
import hashlib, json, os, struct, sys, urllib.request, zlib
from fractions import Fraction
from pathlib import Path
ROOT=Path(__file__).resolve().parent
URL=os.environ.get('PINAKA_S02_URL','https://pinaka-s02-cyber-xo-
x.vercel.app').rstrip('/')+'/sanctuary/verify'

def verify_container(path):
    b=Path(path).read_bytes()
    if b[:8]!=b'PNKVEIL1': raise ValueError('survey magic mismatch')
    version,nimages,nstones=struct.unpack_from('<HHH',b,8)
    if version!=1 or (nimages,nstones)!=(6,7): raise ValueError('unexpected survey version/counts')
    if struct.unpack_from('<I',b,18)[0] != zlib.crc32(b[:18])&0xffffffff: raise ValueError('survey
header CRC mismatch')
    if struct.unpack_from('<I',b,86)[0] != zlib.crc32(b[22:86])&0xffffffff: raise
ValueError('calibration table CRC mismatch')
    if any(b[22:86]): raise ValueError('expected damaged zeroed calibration table')
    return len(b)
```

```
def fr(x):
    q=Fraction(x); return [q.numerator,q.denominator]
def main():
    size=verify_container(ROOT/'recon.survey')
    city=json.loads((ROOT/'city_model.json').read_text())
    ledger=json.loads((ROOT/'supply_records.json').read_text())
    # Recovered from the metric anchor-image observations after correcting the
    # two CRC-torn frames; the stored 2x2 detector table is zeroed in the file.
    D=[[Fraction(6,5),Fraction(1,10)],[Fraction(-1,20),Fraction(11,10)]]
    gardens={g['id']:g for g in city['gardens']}
    template=gardens[1]['template']
    def canon_template(t):
        entries=[]
        for sid in sorted(map(int,t)):
            x,y=t[str(sid)]; fx,fy=Fraction(x),Fraction(y); entries.append(f'{sid}:
{fx.numerator}/{fx.denominator},{fy.numerator}/{fy.denominator}')
        return ';'.join(entries)
    template_digest=hashlib.sha256(canon_template(template).encode()).hexdigest()
    family=[]
    for gid,g in gardens.items():
        if canon_template(g['template'])==canon_template(template): family.append(gid)
    known=[list(e) for e in ledger['roads']]
    sentinel=ledger['corrupt_sentinel']
    bad=[e for e in known if e[2]==sentinel]
    missing=ledger['royal_total_length']-sum(e[2] for e in known if e[2]!=sentinel)
    if len(bad)!=1: raise ValueError('expected exactly one overwritten road')
    repaired=[bad[0][0],bad[0][1],missing]
    edges=[(u,v,missing if w==sentinel else w) for u,v,w in known]
    adj={i:[] for i in range(ledger['n_nodes'])}
    for u,v,w in edges: adj[u].append((v,w)); adj[v].append((u,w))
    dist={ledger['depot_node']:0}; todo=set(adj)
    while todo:
        u=min(todo,key=lambda n:dist.get(n,float('inf'))); todo.remove(u)
        if u not in dist: break
        for v,w in adj[u]:
            if dist[u]+w<dist.get(v,float('inf')): dist[v]=dist[u]+w
    ledger_distance=(ledger['rations_dispatched']-
ledger['rations_returned'])//ledger['rations_per_league']
    matches=[gid for gid in family if dist[ledger['garden_nodes'][str(gid)]]==ledger_distance]
    if len(matches)!=1: raise ValueError(f'expected one fused garden, got {matches}')
    garden_id=matches[0]; garden_distance=dist[ledger['garden_nodes'][str(garden_id)]]
    body={'garden_id':garden_id,'detector':{'d00':fr(D[0][0]),'d01':fr(D[0][1]),'d10':fr(D[1]
[0]),'d11':fr(D[1]
[1])},'template_digest':template_digest,'authentic_family':sorted(family),'repaired_edge':repaired,
'ledger_distance':ledger_distance,'garden_distance':garden_distance}
    print(f'Survey container: {size} bytes; magic/header/table CRCs verified; detector table is
zeroed.')
    print('Recovered detector:',json.dumps(body['detector']))
    print('Authentic family:',body['authentic_family'],'template digest:',template_digest)
    print('Repaired edge:',repaired,'ledger
distance:',ledger_distance,'candidate:',garden_id,'distance:',garden_distance)
    print('Submission:',json.dumps(body,indent=2))
    (ROOT/'submission.json').write_text(json.dumps(body,indent=2)+'\n')
    if '--submit' in sys.argv:
        req=urllib.request.Request(URL,data=json.dumps(body,separators=(',',':')).encode(),headers=
{'Content-Type':'application/json'},method='POST')
        with urllib.request.urlopen(req,timeout=45) as r: result=json.loads(r.read())
        print('Validator:',json.dumps(result,indent=2))
        (ROOT/'transcripts').mkdir(exist_ok=True)
        (ROOT/'transcripts'/'04_validator.txt').write_text(json.dumps(result,indent=2)+'\n')
        if result.get('flag'): print('Flag:',result['flag'])
if __name__=='__main__': main()
```